

AsymptoticAnalysis

A Nonduplicative Technical Audit

Complex-modulus soundness, cancellation, and repair design

Prepared for Vladimir Reshetnikov
10 September 2026

Repository: `VladimirReshetnikov/Asymptotic`
Reviewed revision: `8e859961d7d37f008b826f3a8cad406460271614`

Abstract

This is an incremental source audit against a substantial existing review corpus, not another enumeration of its backlog. The repository's review indexes and consolidated registers cover 36 earlier reports and 231 attributed entries. A broad inspection of core jets, observable operations, specialized asymptotic adapters, interval arithmetic, and Mathics support leads to one independently specified new high-priority soundness finding: the shared absolute-value shortcut infers the reality of an entire finite approximation from the sign of only its leading coefficient. Complex intermediate coefficients can then cancel after the invalid nonlinear rewrite, leaving a real but incorrect result labelled exact.

The report gives source-predicted public witnesses that respect the complex-literal input restriction, exact mathematical countermodels, a fail-closed patch emitter, eight pending Wolfram regression tests, and an independently implemented complex-modulus jet model. The accompanying Python run passes 115 exact/model and patch-fixture checks. These are not Wolfram package tests. No native package execution or native performance improvement is claimed. A positive-real-coordinate modulus extension and a Hermitian convolution prototype provide concrete follow-on development work without recycling the earlier general roadmaps.

Principal conclusion. Validate an operation's mathematical identity *before* allowing later coefficient cancellation to erase evidence that its preconditions failed. For Abs, the minimal sound magnitude repair checks the complete *retained finite jet*; it need not require the unknown remainder itself to be real.

Contents

1	Scope, evidence, and the review result	3
1.1	What was reviewed	3
1.2	Evidence levels	3
1.3	Finding summary	4
2	The relevant internal contract	4
2.1	Jets and uncertainty	4
2.2	The literal-input screen is not a codomain proof	4
2.3	Where the precondition is lost	5
3	ABS-01: the absolute-value shortcut is unsound on complex jets	5
3.1	The implementation	5
3.2	A public witness that respects the input screen	6
3.3	Source-level execution trace	6
3.4	Exact contradiction and order of the error	7
3.5	A polynomial-valued witness with a wrong coefficient sign	7
3.6	Additional consequences and controls	8
4	The minimal repair and its proof	8
4.1	Guard the finite jet at the nonlinear boundary	8
4.2	Why a complete-source reality proof is unnecessary here	8
4.3	Behavior and compatibility of the candidate	9
5	A genuine modulus implementation on the real local coordinate	9
5.1	The intended extension is smaller than complex asymptotics	9
5.2	Normalize the finite approximation	10
5.3	Finite truncation with an explicit logarithmic degree	10
5.4	What the executable reference implements	11
6	A specific performance opportunity: Hermitian pairing	11
7	Validation and reproducibility	12
7.1	What the 115 checks establish	12
7.2	Pending Wolfram tests	13
7.3	Integration sequence and acceptance conditions	13
8	Relationship to earlier reviews and limits of the conclusions	14
8.1	Why this is not a restatement of the preceding findings	14
8.2	Other inspected areas	14
8.3	Conclusions	14
A	Source inspection inventory	15
B	Delivered artifacts and their status	16

1 Scope, evidence, and the review result

1.1 What was reviewed

The source snapshot is the commit printed on the title page. All repository references below are pinned to that revision rather than to a moving branch. The current package name is `AsymptoticAnalysis`; the repository name remains `Asymptotic`. Its modular implementation lives in `src/Kernel`, while the repository-root standalone is generated from those modules. The package's source map identifies the canonical entry point and shared private-module loading model.[1, 2]

The inspection covered portions or all of twenty kernel modules. It included sparse power-log arithmetic and real-coefficient admission; public observable, arithmetic, composition, and refinement paths; the beginning of the exact interval evaluator; Gamma, Barnes, Lambert, and Dirichlet constructions; parameterized and native special-function adapters; Fourier coefficient machinery; and several Mathics compatibility layers. Appendix A records the actual extent. Reading a module is not equivalent to exercising all its paths, and a partial source inspection is not advertised as complete line-by-line verification.

The broad pass was used to identify new, defensible contributions. In particular, the report does not repackage already recorded generic requests for more native coverage, different resource budgets, fuller result schemas, release gates, complex sectors, or general transseries support. Nor does it treat the presence of an earlier repaired defect in an old report as proof that it remains open. The maintained status and intake registers are the main suppression baseline; selected original ledgers and reports nearest to the present finding were also inspected.[5, 6, 7, 8, 9, 10, 11]

1.2 Evidence levels

Three levels of evidence are kept separate throughout this article.

Source-confirmed mechanism.

The displayed implementation of `fwdAbs` checks only the leading coefficient. The observable dispatcher reaches it before the final real-coefficient validator. These are direct properties of the retrieved source.[3, 4]

Exact mathematical and executable-model evidence.

The counterexamples, expansions, symbolic cancellation models, modulus-jet reference implementation, paired convolution, and synthetic patch checks were executed in Python with SymPy. The delivered result file records 115 passed checks and no failed checks, with the complete named case list and runtime versions.

Pending native observation.

The public Wolfram examples, exact diagnostic tags, and the candidate's behavior in a complete package load have not been executed here. The Wolfram service returned connection/502 failures, including on a version probe; a local Mathics installation was also unavailable. The supplied Wolfram scripts are regression artifacts awaiting execution, not transcripts of successful runs.

No complete repository clone was available in the working runtime. Source was read through the GitHub connector; the source excerpt and patch anchor were transcribed from those returned definitions. The patch emitter was tested on isolated text fixtures, not a locally materialized full checkout. This distinction matters because a successful text transformation does not establish successful package loading, standalone regeneration, or compatibility-adaptor installation.

1.3 Finding summary

ID	Priority	New contribution
ABS-01	High	A complex finite argument is treated as real by the shared Abs sign shortcut. Subsequent cancellation can produce a wrong, real, zero-remainder result.

This is deliberately one finding, not several inflated counts for the same root cause. The conjugate-pair sum, squared-modulus sum, and zero-producing subtraction are different witnesses and consequences of the same defect. The proposed complex-modulus implementation and paired-convolution optimization are *development proposals*, not assertions that two more unrelated bugs were found.

A high priority is appropriate even without an observed native transcript: the mathematical primitive has a direct counterexample, the public control flow is short, and its predicted result has no remainder to warn a downstream consumer. The uncertainty is about unexecuted whole-package behavior, not about the falsity of the underlying identity.

2 The relevant internal contract

2.1 Jets and uncertainty

The ordinary precision algebra represents a local quantity by

$$J = (T, P, D), \quad T = \{(\alpha_j, Q_j(L))\}_{j=1}^m, \quad L = \log u, \quad u \rightarrow 0^+.$$

Its finite approximation is

$$p(u) = \sum_{j=1}^m u^{\alpha_j} Q_j(\log u),$$

and finite P records a magnitude remainder

$$z(u) = p(u) + O\left(u^P (1 + |\log u|)^D\right).$$

An infinite precision marker represents an exact finite expression. The core implements addition, multiplication, powers, logarithms, exponentials, and composition in this representation.[3]

The distinction between *intermediate* and *completed* coefficients is important. The source explicitly permits complex intermediate summands that cancel, and calls `realCoefficientRows` at representation boundaries. The implementation of `seriesMake` performs that final validation before building a public result. This is a reasonable policy for algebraic collection: for example, $(1 + az) + (1 - az) = 2$ is a valid identity regardless of whether a is real.[3, 4]

It is not, by itself, a policy for applying nonlinear functions to those intermediates. A nonlinear rewrite needs its own precondition. One cannot establish that a nonlinear step was valid merely by observing that the ultimate coefficient vector is real.

2.2 The literal-input screen is not a codomain proof

The source's initial exact-input predicate is syntactic:

```
exactQ[e_] := FreeQ[e, _Real | _Complex];
validateInput[f_, limit_] := (
  If[! exactQ[f], fail["InexactInput", ...]];
  ...);
```

It rejects literal complex-number atoms, including `I`, as well as approximate real numbers. It does not prove that every symbolic intermediate is real. A symbolic parameter `a` with the exact constraint `a^2 == -1` contains neither a `Real` nor a `Complex` atom. The input screen and the later coefficient-reality proof serve different purposes.[3]

Consequently, the most tempting illustrative call with literal `I` is *not* the public reproducer used in this report. That call is retained as a negative control in the regression file. The actual proposed witness uses a symbolic parameter, and its overall observable is real-valued on the retained real input path.

2.3 Where the precondition is lost

The relevant observable flow is

$$\begin{aligned} \text{SeriesObservable} &\longrightarrow \text{seriesJetApply} \longrightarrow \text{fwdAbs} \\ &\longrightarrow \text{pAdd/pMul} \longrightarrow \text{seriesMake}. \end{aligned}$$

In `seriesJetApply`, `Abs` has a dedicated arm:

```
h === Abs,
  fwdAbs[seriesJetApply[e[[1]], x, input, d, cut, limit],
    ell, ass]
```

It does not take the generic unary Taylor arm, where the current source checks complete-argument reality, approach compatibility, and returned Taylor data. The public wrapper validates the input expression, handles conditional expressions, applies this dispatcher, and then builds the result. It does not first replace the whole observable by a complex-modulus formula.[4]

That difference also explains the novelty relative to the preceding Taylor reviews: no native Taylor oracle is needed to trigger the present mechanism. An exact monomial input and finite algebraic intermediate jets suffice.

3 ABS-01: the absolute-value shortcut is unsound on complex jets

3.1 The implementation

The complete reviewed helper is small enough to show:

```
fwdAbs[j : {T_, P_, D_}, ell_, ass_] := Module[{q, c, degree},
  If[T === {}, Return[j, Module]];
  q = T[[1, 2]]; degree = polyDegree[q, ell];
  c = (-1)^degree Coefficient[q, ell, degree];
  Which[provablyPositive[c, ass], j,
    provablyNegative[c, ass], pScale[j, -1, ell, ass],
    True, fail["UnprovedSign",
      "The eventual sign of the absolute-value argument could not be proved."]]];
```

This is `src/Kernel/AsymptoticAnalysis.wl`, lines 460–465 in the pinned snapshot.[3]

For a *real* finite jet, the leading logarithmic coefficient determines an eventual sign, since $L \rightarrow -\infty$. Thus the factor $(-1)^d$ is sensible. The missing hypothesis is that the complete finite approximation is real. A real positive leading coefficient does not supply that hypothesis. For instance, $1 + iu$ has leading coefficient 1 but

$$|1 + iu| = \sqrt{1 + u^2} \neq 1 + iu.$$

The complex meaning of `Abs` is the ordinary modulus, not a formal real-sign operator.[12]

3.2 A public witness that respects the input screen

Run the following in a fresh kernel with the pinned package loaded:

```
Clear[x, z, a];
s = AsymptoticAnalysis`AsymptoticExpansion[
  x, {x, 0, 4},
  Assumptions -> a^2 == -1,
  "Backend" -> "Package"];

r = AsymptoticAnalysis`SeriesObservable[
  s, Abs[1 + a z] + Abs[1 - a z], z,
  "Cutoff" -> 4];

{Normal[r], r["Remainder"], r["Exact"], r["Assumptions"]}
```

Source-predicted behavior, not a recorded native output: the finite expression becomes 2, the remainder becomes zero, and the exactness flag becomes true. The parameter condition remains a condition on a , not a proof that a is real.

The symbolic condition is satisfiable over the complex numbers and says $a \in \{i, -i\}$. Both values give exactly the same real observable on real x :

$$F(x) = |1 + ax| + |1 - ax| = 2\sqrt{1 + x^2}. \quad (1)$$

The source input is $s(x) = x$, a real exact function; the observable F is also real and nonnegative. This is not a request for a complex output series or an expansion in a complex sector.

The constraint is useful because it removes any ambiguity about the intended parameter domain. An unconstrained a would also fail to justify the shortcut, but a user or reader might mistakenly infer an unstated real-parameter policy. Here, the realness of the *observable* is true while the realness of its *arguments to each modulus* is explicitly false away from zero.

3.3 Source-level execution trace

The ordinary exact input gives the internal jet

$$J_x = (\{(1, 1)\}, \infty, 0).$$

Before `Abs`, the two inner expressions give

$$J_+ = (\{(0, 1), (1, a)\}, \infty, 0), \quad J_- = (\{(0, 1), (1, -a)\}, \infty, 0).$$

Their leading polynomial is 1. The reviewed helper therefore returns J_+ and J_- unchanged. Addition then cancels the a -dependent blocks:

$$J_+ + J_- = (\{(0, 2)\}, \infty, 0).$$

The final real-coefficient validator has nothing nonreal left to reject. `seriesMake` associates an infinite remainder precision with zero remainder and exactness. Every algebraic step *after* the false `Abs` rewrite is internally consistent; that is why a stronger final output-type check alone would not repair the problem.[3, 4]

This trace is also a useful debugging guide. Instrument the argument and return value of `fwdAbs`, not just the final `GeneralizedSeries`. The loss of information occurs before the evidence becomes invisible through cancellation.

3.4 Exact contradiction and order of the error

The correct local expansion of (1) is

$$F(x) = 2 + x^2 - \frac{x^4}{4} + \frac{x^6}{8} - \frac{5x^8}{64} + O(x^{10}). \quad (2)$$

For the exclusive cutoff 4, an ordinary sound answer retains $2 + x^2$ and an appropriate fourth-order remainder. A structured refusal is also sound if this coefficient class is not implemented. The predicted exact constant 2 is not.

There is no numerical conditioning issue here. Rationalization gives the exact identity

$$F(x) - 2 = \frac{2x^2}{\sqrt{1+x^2} + 1},$$

so

$$\lim_{x \rightarrow 0^+} \frac{F(x) - 2}{x^2} = 1, \quad \lim_{x \rightarrow 0^+} \frac{F(x) - 2}{x^4} = +\infty.$$

Thus even attaching $O(x^4)$ to the constant 2 would not repair the missing coefficient. At the convenient exact point $x = 3/4$, $F(x) = 5/2$, whereas the predicted constant is 2. The local limit, rather than this single point, is the asymptotic proof; the rational point is an easily checked diagnostic.

3.5 A polynomial-valued witness with a wrong coefficient sign

A second observable eliminates even the square root from the exact answer:

```
r2 = AsymptoticAnalysis`SeriesObservable[
  s, Abs[1 + a z]^2 + Abs[1 - a z]^2, z,
  "Cutoff" -> 4];
```

On the retained parameter condition,

$$|1 + ax|^2 + |1 - ax|^2 = 2 + 2x^2.$$

The same shortcut instead propagates

$$(1 + ax)^2 + (1 - ax)^2 = 2 + 2a^2x^2 = 2 - 2x^2.$$

The resulting wrong coefficients are real; the predicted error is exactly $4x^2$. This witness is valuable because neither the correct expression nor the incorrectly rewritten one needs an infinite Taylor series.

Both witnesses should be kept: the first detects a lost term and false exactness, while the second detects a real coefficient of the wrong sign. They are two tests for ABS-01, not two separately counted findings.

3.6 Additional consequences and controls

Subtracting 2 from the first predicted result produces an exact zero in place of a genuinely nonzero quadratic germ. Downstream arithmetic is then entitled to eliminate this term or to report that it has no nonzero leading block. Those consequences are caused by the original false exactness; they should not be blamed on otherwise correct zero handling.

The difference

$$|1 + ax| - |1 - ax| = 0$$

provides a complementary symptom. The erroneous helper produces $2ax$, which can be rejected by the final real-coefficient check despite the correct observable being identically zero. This shows why testing only isolated `Abs` calls, where a nonreal output may be caught later, does not exercise the most dangerous cancellation case.

The regression catalog also retains positive controls. Under `Element[a, Reals]`, the first pair really is eventually equal to 2. The functions $|1 + x|$ and $|-1 + x|$ have the expected real linear germs. Ordinary algebraic cancellation without a nonlinear rewrite remains valid. The literal-I spelling is expected to fail at input validation and is explicitly not counted as a bug reproduction.

4 The minimal repair and its proof

4.1 Guard the finite jet at the nonlinear boundary

A conservative repair is to retain the fast sign implementation but establish that every retained coefficient polynomial is real under the recorded assumptions. The supplied emitter inserts the following check after the empty-jet case and before the leading-sign computation:

```
If[! AllTrue[T,
  TrueQ[realPolynomialCondition[#[[2]], ell, ass]] &],
  fail["UnprovedAbsJetRealness",
    "The absolute-value sign shortcut requires every retained coefficient to be
    provably real.",
    <|"FiniteJet" -> T, "Assumptions" -> ass|>]];
```

The existing coefficient predicate is already available in the core. The proposal does not install a global restriction against all complex intermediates, and it does not change `pConst`, `pAdd`, or legitimate algebraic cancellation. It repairs the precondition at the operation that requires it.[3]

The new failure name is a proposed diagnostic, not an existing upstream API promise. It identifies the exact reason the real-sign fast path was refused. The full finite jet is included for diagnostic inspection; a production version may prefer a compact offending-block payload, especially for large jets.

4.2 Why a complete-source reality proof is unnecessary here

A stronger-looking fix would require the full argument of `Abs`, including its unknown error, to be real. That requirement is sufficient, but is not necessary for a magnitude approximation.

Theorem 4.1 (Sound real-finite-jet shortcut). *Let $p(u)$ be a finite real power-log expression on a positive*

real local coordinate. Suppose its first nonzero block establishes an eventual sign $\sigma \in \{-1, 1\}$. Let

$$z(u) = p(u) + R(u), \quad |R(u)| \leq Cu^P(1 + |\log u|)^D$$

on a sufficiently small deleted neighborhood. The error R may be complex. Then

$$|z(u)| = \sigma p(u) + O(u^P(1 + |\log u|)^D).$$

Proof. The eventual sign gives $|p(u)| = \sigma p(u)$. The reverse triangle inequality in $\mathbb{C}$ yields

$$||z(u)| - |p(u)|| \leq |z(u) - p(u)| = |R(u)|.$$

Combining these two statements proves the assertion with the same magnitude majorant constant. $\square$

The theorem explains the appropriate granularity of the guard. Reality must hold for the complete *retained approximation*, not just its leading coefficient. But discarded complex information already covered by the recorded error does not force a refusal. This matters when a later refinement first reveals a previously omitted imaginary block: the weaker earlier magnitude approximation can have been valid even though the refined sign-only path must now refuse or use a true modulus expansion.

The empty-jet case is also sound for magnitudes: $z = O(R)$ implies $|z| = O(R)$. The candidate preserves it. This is not a theorem about arbitrary derivative contracts through a zero-crossing modulus. The supplied reference model intentionally carries no derivative certificates, and the candidate should not be presented as a general theorem about differentiating nonsmooth observables.

4.3 Behavior and compatibility of the candidate

On ABS-01, the guard fails before a and $-a$ can cancel. The proposed minimal implementation therefore returns a structured refusal, not the mathematically richer expansion $2 + x^2 + O(x^4)$. That is an intentional soundness-first repair. Restoring support for this valid real observable is separate work, specified in Section 5.

For a real finite jet, the original sign logic remains unchanged. The cost is one retained-coefficient reality pass at each Abs node. A later optimization can reuse already established coefficient facts within the same assumptions scope, but this audit does not claim a measured performance regression or improvement from the guard.

The emitter requires one exact copy of the reviewed function and refuses an absent, duplicated, modified, or already-patched anchor. It reads its input and writes a unified diff to standard output. It does not mutate a checkout or claim that matching one function establishes the revision identity of an entire repository.

5 A genuine modulus implementation on the real local coordinate

5.1 The intended extension is smaller than complex asymptotics

The useful next capability is not an unspecified complex-sector engine. The coordinate remains $u \rightarrow 0^+$, all exponents remain real, and the completed output remains real. Only intermediate finite coefficients are allowed to be complex, followed by the well-defined map $z \mapsto |z|$.

This directly supports real observables built from complex algebraic factors, Euclidean norms, and conjugate pairs. It also avoids the unsatisfactory user experience in which $\sqrt{1 + x^2}$ is supported while an

equivalent modulus expression is either refused or incorrectly treated as a real sign. These are specific consequences of ABS-01 rather than a repetition of the repository's existing general complex-extension roadmap.

5.2 Normalize the finite approximation

Assume the leading coefficient is a fixed, nonzero complex constant, independent of L . Write

$$p(u) = cu^\alpha(1 + U(u)), \quad c \neq 0,$$

where U is a finite complex power-log expression with positive valuation. Set

$$H = U + \overline{U} + U\overline{U}. \quad (3)$$

Then H is real, $H \rightarrow 0$, and

$$|p(u)| = |c|u^\alpha\sqrt{1 + H(u)}. \quad (4)$$

The eventual positive square-root branch is automatic from $H \rightarrow 0$; it is not an arbitrary branch selection.

Conjugation here fixes the real coordinate and $L = \log u$. It conjugates coefficient scalars only. Substituting an unrelated complex conjugation rule for u , or treating L as a free complex variable, would solve a different problem.

The simplest witness has $c = 1$, $U = iu$, and $H = u^2$. In particular, the imaginary first-order perturbation disappears from $U + \overline{U}$, but reappears in the *real quadratic* product $U\overline{U}$. That is precisely the contribution the reviewed sign shortcut loses.

5.3 Finite truncation with an explicit logarithmic degree

Let the requested absolute exponent cutoff be h , and let the incoming magnitude error be $O(u^P \Lambda^D)$, where $\Lambda = 1 + |\log u|$. Work only to

$$h_* = \min(h, P)$$

when P is finite; otherwise take $h_* = h$. Suppose $h_* > \alpha$, and suppose the finite expression H has positive valuation v and maximum coefficient logarithmic degree d . Define

$$K = \left\lceil \frac{h_* - \alpha}{v} \right\rceil.$$

Compute

$$|c|u^\alpha \sum_{k=0}^{K-1} \binom{1/2}{k} H^k \quad (5)$$

and retain only blocks of absolute exponent strictly below h_* .

Theorem 5.1 (Conservative modulus truncation). *Under these hypotheses, the retained expression $M(u)$ satisfies*

$$|z(u)| - M(u) = O(u^P \Lambda^D) + O(u^{h_*} \Lambda^{Kd}),$$

with the first term omitted when the incoming finite approximation is exact. The coefficients in M are real.

Proof. Since $H \rightarrow 0$, the real function $\sqrt{1+H}$ has a Taylor remainder $O(H^K)$ after degree $K-1$. Each block of H has exponent at least v and logarithmic degree at most d , so

$$H = O(u^v \Lambda^d), \quad u^\alpha H^K = O(u^{\alpha+Kv} \Lambda^{Kd}) = O(u^{h_*} \Lambda^{Kd}).$$

Every discarded finite product from (5) has exponent at least h_* and logarithmic degree at most $(K-1)d$, so it obeys the same conservative bound. The reverse triangle inequality transports the incoming error from z to $|p|$ without any division by p . Finally, H is real and its binomial coefficients are real, proving coefficient reality. $\square$

The formula is intentionally conservative. It does not infer exactness because the currently visible omitted coefficient vanishes. Sharper first-omitted-block bounds can be added with an independent finite-support or tail argument, but are not needed to repair ABS-01.

When $h \leq \alpha$, no leading block is requested; $|p| = O(u^\alpha)$ supplies a valid magnitude answer. When $H = 0$ exactly, $|p| = |c|u^\alpha$ is an exact finite modulus of the finite approximation, while any original error must still be retained. The all-real finite-jet path can continue to use Theorem 4.1, including real logarithmic leading polynomials.

5.4 What the executable reference implements

The delivered code/modulus_jet.py implements this magnitude construction with sparse dictionaries, exact rational exponents, and SymPy polynomials in a real symbol L . Its coefficient scalars must be exact numeric expressions; it deliberately refuses unproved symbolic-parameter cases rather than attempting to reproduce Wolfram's assumption prover.

It supports real finite jets by eventual sign, complex log-independent leading coefficients by (4), logarithmic perturbations, inherited magnitude errors, exact zero, and cutoff clipping. It refuses unsupported complex logarithmic leading amplitudes, inexact constants, inconsistent retained/error orders, and exhausted operation budgets. It is neither a replacement package nor a verified translation of all upstream jet operations.

A nonconstant complex leading logarithmic polynomial is a meaningful further case, but its norm need not be polynomial in L . For example, $|1 + iL| = \sqrt{1 + L^2}$. Supporting it requires another normalization or a richer coefficient class. The prototype refuses this case explicitly; it does not silently project its leading coefficient onto the real axis.

The reference initially forms the full finite product $U\bar{U}$ subject to its budget. That is an intentionally simple implementation choice, not a claim that it has solved the repository's previously recorded general truncation-performance issues.

6 A specific performance opportunity: Hermitian pairing

For a finite expansion

$$p(u) = \sum_{j=1}^m u^{\alpha_j} c_j(L),$$

forming $p\bar{p}$ naively considers m^2 ordered coefficient products. The involution exchanging the two factors gives the identity

$$p\bar{p} = \sum_i u^{2\alpha_i} |c_i|^2 + 2 \sum_{i < j} u^{\alpha_i + \alpha_j} \operatorname{Re}(c_i \bar{c}_j). \quad (6)$$

Thus the coefficient-product enumeration can use the upper triangle, with $m(m + 1)/2$ pairs, while collecting equal summed weights normally.

The separate `hermitian_square` prototype implements this identity and was compared with the ordinary ordered convolution. These are measured *operation counts inside the Python model*, not native timing results:

Input support m	Ordered products	Paired products
8	64	36
16	256	136
32	1024	528
64	4096	2080

All four resulting exact polynomials matched. At support 64, the prototype performs 2016 fewer candidate coefficient products. Both algorithms remain quadratic, and real-part simplification has its own cost. These counts therefore justify trying the specialization, not promising an approximately twofold wall-clock speedup in Wolfram or Mathics.

An integrated implementation should pair before coefficient simplification, use the same exact exponent-equality machinery as the ordinary jet algebra, and only then collect equal resulting weights. One-sided complex coefficient cancellation must not be confused with cancellation of independent unknown errors: (6) concerns the *finite approximation*, whose algebraic correlations are known.

This optimization is particularly natural at the new modulus boundary. It is not a request to rewrite all multiplication in the package. The delivered prototype is separate from the main modulus routine so that its exact output and product-count effect can be inspected without implying an already integrated upstream optimization.

7 Validation and reproducibility

7.1 What the 115 checks establish

The executed checks include the two symbolic source-control-flow models and their exact mathematical contradictions; a grid of imaginary perturbations with rational exponents and several cutoffs; equality below the requested cutoff against independent SymPy expansions; exact phase and conjugation invariance; real, logarithmic, zero, and pure-error cases; input-precision clipping; explicit unsupported-input and budget refusals; Hermitian convolution equality and counts; and the patch emitter’s fail-closed behavior.

The run is reproducible from the archive root:

```
python code/independent_checks.py \
--output evidence/independent_results.json
```

The JSON records the complete check list rather than only a success total. `evidence/independent_checks.txt` records the authoring run’s summary. The dependency is declared in `code/requirements.txt`; Python and SymPy versions used for that run are stored in the JSON.

The case count is not a statistical claim, a formal proof of a general CAS, or an upstream test count. Several checks deliberately share a mathematical family. The general error statements are supplied by the proofs in this article; the executable cases test their implementation and boundary behavior.

7.2 Pending Wolfram tests

The eight tests in `AbsGuardRegressions.wlt` specify the conservative candidate’s desired behavior, not an already accepted upstream contract. The first two require the new structured refusal; the remaining six check real-parameter, signed-real, cancellation, literal-input, and exact-zero controls. A later full modulus implementation should replace the first two refusals with correct coefficient and remainder assertions, not retain them as permanent feature limitations.

To observe the baseline in a fresh kernel:

```
wolframscript -file code/characterize.wl \
  /checkout/Asymptotic/AsymptoticAnalysis.wl
```

The script prints the kernel version, selected input entry, seed result, observable results, and independent identities. A failed seed construction terminates the script rather than pretending to have reached the target helper. The exact message stream may differ from the source prediction; that is why the observation script is separate from the desired-contract suite.

To emit the candidate diff:

```
python code/abs_guard_patch.py \
  /checkout/Asymptotic/src/Kernel/AsymptoticAnalysis.wl \
  > abs-guard.patch
```

Review and apply that diff only to an appropriate scratch checkout. The upstream source map states that the standalone must be regenerated from the modular sources.[2]

```
python validation/build_standalone.py
python validation/build_standalone.py --check
wolframscript -file /path/to/review/code/run_focused.wl \
  /checkout/Asymptotic/AsymptoticAnalysis.wl
```

The driver requires all eight desired-contract tests to succeed for a zero exit status. It has not been executed in a native kernel here. Nothing in this archive is an instruction to run the upstream full suite contrary to the repository’s current development guidance.

7.3 Integration sequence and acceptance conditions

First characterize the pinned baseline and preserve its actual output. This is especially useful for checking whether a surrounding evaluator normalization changes the proposed public witness before it reaches the helper. If so, test the helper invariant directly as well; do not replace a failed public reproduction with an invented transcript.

Second, apply the finite-jet guard and establish that the two dangerous cancellation cases either refuse at the intended boundary or return a mathematically correct result through an independently justified path. Real controls must remain valid. Test modular and regenerated standalone entries in fresh processes so that earlier definitions do not mask the edited helper.

Third, implement the true modulus operation as a distinct semantic operation and rerun the same witness family as positive functionality tests. The input constraint $a^2 = -1$ is a useful parameter-proof case in Wolfram; the Python prototype’s $a = i$ specialization is only its independent mathematical oracle. A runtime

that cannot prove the required coefficient facts should preserve a clear refusal rather than accepting them numerically.

Finally, evaluate the paired convolution in the actual interpreter with representative coefficient simplification costs. The exact pair-count reduction is already established; the native cost model remains to be measured. These are bounded acceptance steps tied to ABS-01, not a repeated general project-wide validation roadmap.

8 Relationship to earlier reviews and limits of the conclusions

8.1 Why this is not a restatement of the preceding findings

The nearest earlier entries concern final coefficient reality, the real-domain premises of some operations, and generic native observable Taylor admission. The present source already contains the final real-coefficient guard and the strengthened generic Taylor arm.[5, 6, 4]

ABS-01 survives those protections for two specific reasons: Abs does not use the generic unary arm, and the invalid intermediate terms cancel to a *real* final coefficient vector. No inaccurate native order, missing Taylor coefficient, opaque custom function, incomplete native result, or rounded numerical test is involved. The needed repair is the precondition of this shared modulus shortcut, not another final-output realness check.

The selected original ledgers identify their related findings as constant coefficient admission, discarded complex information affecting a trigonometric bound, and generic Taylor order/approach behavior. They were used to delimit the new mechanism, not copied into the new finding list.[8, 9, 10, 11] The novelty ledger in the archive records these relationships. The review does not claim to have reread every paragraph of all 36 archived articles; it uses their consolidated inventories and a targeted original-source cross-check.

8.2 Other inspected areas

The surrounding inspection was useful even where it did not yield another new finding. The Barnes logarithm module explicitly attaches a finite Bernoulli tail and uses positive-argument recurrence reductions. The Dirichlet adapter contains an integral-comparison Zeta bound and a summed Taylor-moment Lerch bound. The interval evaluator uses exact rational directed rounding in its arithmetic primitives. These implementation facts informed the review scope; they are not presented as new certifications of those subsystems.[13, 14, 15]

Likewise, reading Mathics-specific assumption, algebra, Taylor, and call adapters does not establish parity with Wolfram. The service failures prevented a fresh runtime comparison. Existing compatibility issues recorded in the intake register were not counted again merely because their code appeared in the inspection.[7, 2]

Some private-helper concerns would require an additional violated call-site precondition to become a demonstrated public defect. They were not promoted to findings just to lengthen the report. The resulting small finding count reflects the explicit nonduplication requirement and the evidence threshold, not a claim that the rest of the repository is bug-free.

8.3 Conclusions

The immediate change is small: reject a nonreal retained finite jet before using the real-sign implementation of Abs. The mathematical reason is also small, but important: the sign of a leading coefficient is not a codomain proof for all retained terms, and final cancellation cannot repair an invalid nonlinear identity.

The constructive continuation is well specified. Compute the modulus of the finite approximation through its Hermitian norm, carry the original error by the reverse triangle inequality, and bound the additional finite truncation with an explicit logarithmic degree. This produces real coefficients on the same positive real coordinate, without promising a general complex asymptotic engine. The paired convolution offers a concrete, exactly checked implementation optimization for that operation.

The delivered patch is a candidate; the independent model is executed; the Wolfram regressions remain pending. Keeping those three statements separate is part of making the result actionable rather than overstated.

A Source inspection inventory

Module under src/Kernel	Extent and purpose
AsymptoticAnalysis.wl	Relevant sections: input validation, coefficient proofs, jet algebra, forward operations, native tail import.
SeriesOperations.wl	Extensive sections: representation, observable dispatch, composition scope, derivatives, refinement; not every line.
SeriesArithmetic.wl	Initial arithmetic, held normalizer, and dispatch sections.
SeriesEnvelopeArithmetic.wl	Initial approach, error-normalization, and envelope-construction sections.
InverseCertificates.wl	Initial interval primitives, expression enclosure, domain checks, and seed logic.
GammaInverse.wl	Model admission, ordered coefficients, remainder construction, and result assembly sections.
BarnesInverse.wl	Complete retrieved module.
BarnesForward.wl	Complete retrieved module.
LambertInverse.wl	Core extraction, construction, metadata, and residual sections; one intervening construction segment not retrieved.
DirichletSpecialFunctions.wl	Complete retrieved module.
ParameterizedSpecialFunctions.wl	Complete retrieved module.
SpecialFunctionAdapters.wl	Erfc, Gamma, and threshold adapter sections.
NativeSpecialFunctions.wl	Initial native-tree and sector-processing sections.
FourierCoefficients.wl	Initial coefficient algebra, convolution, source parsing, and composition sections.
MathicsAssumptions.wl	Complete retrieved module.
MathicsAlgebra.wl	Complete retrieved module.
MathicsTaylor.wl	Complete retrieved module.
MathicsSpecialFunctions.wl	Complete retrieved module.
MathicsSimplification.wl	Complete retrieved module.
MathicsCalls.wl	Complete retrieved module.

“Complete retrieved module” describes reading scope, not formal verification or a successful runtime load. Repository overview, source-map documentation, review indexes, consolidated intake registers, selected original review ledgers, and the upstream license were also retrieved. The archive contains a compact machine-readable inspection inventory, not a mirror of the repository.

B Delivered artifacts and their status

Artifact	Status and purpose
article/audit.tex, article/audit.pdf	This article and its compiled rendering.
article/build.sh	Rebuilds the PDF with an installed TeX distribution.
code/abs_guard_patch.py	Executed on isolated text fixtures; emits a candidate unified diff, does not edit a checkout.
code/modulus_jet.py	Executed independent magnitude model and Hermitian convolution prototype.
code/independent_checks.py	Executed exact/model test runner; 115 passed checks in the delivered record.
code/AbsGuardRegressions.wlt	Eight desired-contract Wolfram tests; not executed here.
code/characterize.wl	Pending native baseline observation script.
code/run_focused.wl	Pending native regression driver.
evidence/independent_results.json	Actual independent run, case list, versions, and operation counts.
evidence/novelty_ledger.csv	Distinguishes ABS-01 and its proposed extensions from nearby prior items.
evidence/inspection_inventory.json	Actual source-reading scope.
source-excerpts/fwdAbs.baseline.wl	Transcribed reviewed helper used as the exact patch anchor.

No checksum files, font files, repository modifications, or fabricated native logs are included. Commit identifiers in the article are source provenance.

References

- [1] AsymptoticAnalysis repository overview, pinned revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/README.md>.
- [2] Modular kernel implementation and validation source map. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/README.md>.
- [3] Core implementation, especially lines 172–176 (input screen), coefficient-boundary helpers, and lines 460–465 (fwdAbs). <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/AsymptoticAnalysis.wl>.
- [4] Explicit series operations, especially seriesMake, seriesJetApply, and SeriesObservable. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/SeriesOperations.wl>.
- [5] Consolidated code-review status register. https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/docs/development/CODE_REVIEW_STATUS.md.
- [6] Wave-three review intake and attribution register. https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/docs/development/WAVE_3_INTAKE.md.
- [7] Wave-four review intake and attribution register. https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/docs/development/WAVE_4_INTAKE.md.
- [8] Prior review 7, original finding ledger, particularly F04. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/external-reports/code-review/wave-1/code-review-7/evidence/findings.csv>.

- [9] Prior review 12, novelty ledger, particularly N02. https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/external-reports/code-review/wave-2/code-review-12/evidence/novelty_ledger.csv.
- [10] Prior review 22, precision-boundary audit and evidence scope. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/external-reports/code-review/wave-3/code-review-22/README.md>.
- [11] Prior review 25, incremental technical review and evidence scope. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/external-reports/code-review/wave-3/code-review-25/README.md>.
- [12] Wolfram Research, *Abs: Absolute Value (Modulus)*, official Wolfram Language documentation, accessed 10 September 2026. <https://reference.wolfram.com/language/ref/Abs.html>.
- [13] Barnes forward logarithmic construction. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/BarnesForward.wl>.
- [14] Dirichlet special-function constructions and remainder bounds. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/DirichletSpecialFunctions.wl>.
- [15] Exact rational inverse-certificate arithmetic. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8e859961d7d37f008b826f3a8cad406460271614/src/Kernel/InverseCertificates.wl>.